PIXEL SHOT: HIGH-SPEED GUN ACTION PLATFORMER

1. Project Overview

Pixel Shot is a high-speed, pixel-art 2D action platformer developed entirely in Python using the Pygame library. Players control a gunslinger character who must fight through handcrafted levels using precise shooting, aggressive dashing, and a unique bullet-block mechanic — shooting incoming enemy bullets out of the air with perfectly timed shots of their own.

The game is inspired by the pace and intensity of Katana Zero but replaces melee combat with a fully ranged, gun-based combat system. The core loop rewards accuracy, spatial awareness, and fast decision-making. Every level is a one-hit-kill gauntlet that demands mastery of movement and Marksmanship.

Key highlights of the project:

• Gun-based one-hit-kill combat with a bullet-block (bullet vs. bullet) interception mechanic.
• Pixel-art aesthetic with fluid sprite animations and parallax scrolling backgrounds.
• Dash ability for rapid repositioning and brief invincibility frames.
• Score and time-based ranking system (S/A/B/C/D) per level.
• Persistent session statistics tracked and exported to CSV for data analysis.
• 100% Python implementation using Pygame — no external game engine required.

2. Project Review

Reference Project: Katana Zero (Askiisoft, 2019)

Katana Zero is a 2D action-platformer built around melee combat, slow-motion, and one-hit-kill gameplay. Players slash enemies and physically deflect projectiles back using a sword. It is commercially released and praised for its precise game feel, tight controls, and strong visual identity. Planned Improvements and Differentiations:

• **Combat System:** Unlike Katana Zero's melee slash-and-deflect, Pixel Shot replaces this entirely with gun combat — all offense and defense is ranged.

• **Bullet-Block Mechanic:** Instead of reflecting bullets with a sword, the player shoots incoming enemy bullets to destroy them mid-air. This requires timing, aim, and ammo management.

• **Python & Pygame Stack:** The entire game — engine, physics, rendering, input, and data logging — is built in Python, consistent with the data analysis pipeline.

• **Statistical Layer:** Pixel Shot logs granular gameplay data per session (shots fired, accuracy, dashes, deaths), feeding a Python-based analysis dashboard using pandas and matplotlib.

• **Modular OOP Design:** A clean Python class hierarchy makes levels, enemies, and projectiles easy to extend and maintain without touching unrelated systems.

3. Programming Development

3.1 Game Concept

The player controls a lone gunslinger moving through dangerous environments: industrial corridors, open rooftops, and underground vaults. The objective of each level is to reach the exit alive, defeating or evading all enemies along the way.

Core Mechanics:

• **One-hit death:** A single enemy bullet kills the player instantly, creating high-stakes tension throughout every run.

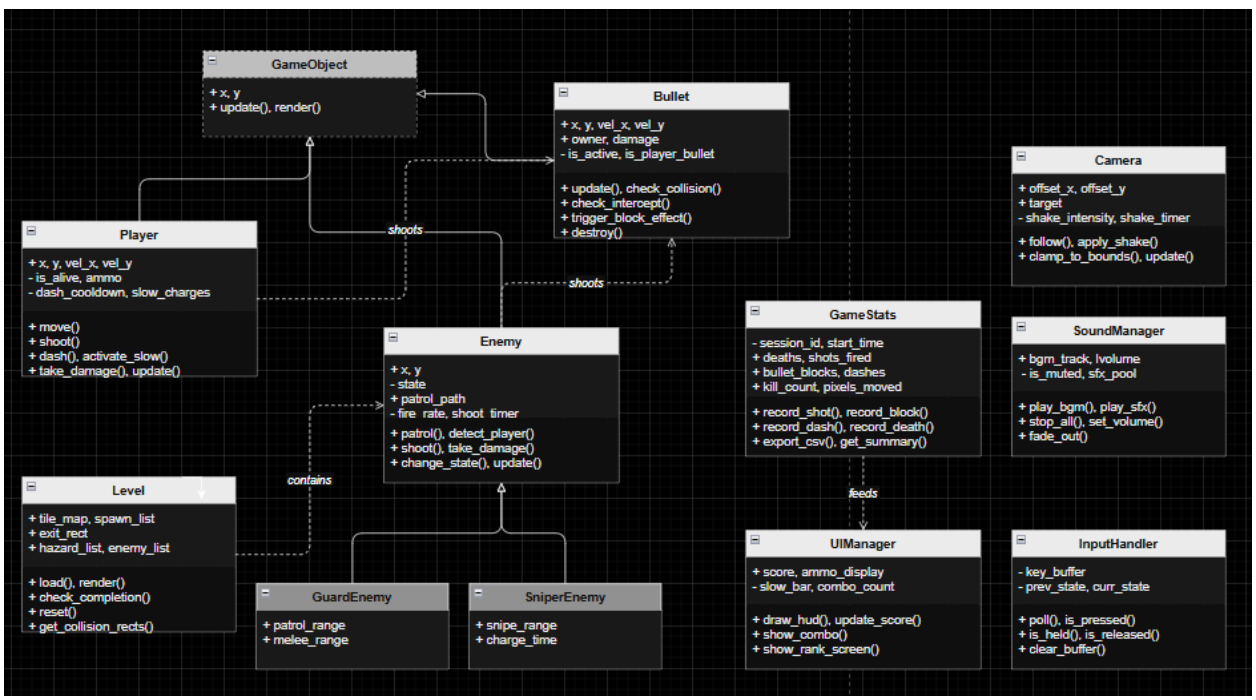
• **Shoot:** The player fires bullets in the aimed direction. Bullets travel at high speed and kill enemies on contact.
• **Bullet-Block:** When an enemy bullet is incoming, the player can shoot it to destroy it mid-flight. A successful intercept requires firing at the correct moment and angle.
• **Dash:** The player dashes horizontally at high speed, gaining brief invincibility frames. Dash has a short cooldown and is the primary evasion tool.
• **Time Slow:** A limited slow-motion ability that reduces game speed to 25%, giving the player time to aim bullet-blocks or plan movement through tight situations. Winning Condition: reach the level exit. The final score is calculated from completion time, enemy kills, bullet-blocks landed, and deaths. Each level awards a rank from S (best) to D (lowest).

3.2 Object-Oriented Programming Implementation

Class	Role	Key Attributes	Key Methods
Player	Represents the player character. Handles movement, shooting, dashing, and slow-time activation.	x, y, vel_x, vel_y, is_alive, dash_cooldown, slow_charges, ammo	move(), shoot(), dash(), activate_slow(), take_damage(), update()
Enemy	Base class for all enemy types. Handles AI state and combat behaviour. Subclassed per enemy variant.	x, y, state, patrol_path, fire_rate, shoot_timer	patrol(), detect_player(), shoot(), take_damage(), change_state(), update()
Bullet	Represents any projectile in the game — player shots and enemy shots alike.	x, y, vel_x, vel_y, owner, damage, is_active	update(), check_collision(), destroy()
BulletBlock	Detects and resolves collisions between a player bullet and an incoming enemy bullet.	player_bullet, enemy_bullet, resolved	check_intercept(), resolve(), trigger_effect()
Level	Loads and manages the tilemap, enemy spawns, hazards, and exit trigger.	tile_map, spawn_list, exit_rect, hazard_list, enemy_list	load(), render(), check_completion(), reset(), get_collision_rects()

GameStats	Records all statistical data for a single play session in memory, then exports to CSV.	session_id, start_time, deaths, shots_fired, bullet_blocks, dashes, kill_count, pixels_moved	record_shot(), record_block(), record_dash(), record_death(), export_csv(), get_summary()
UIManager	Draws all HUD elements: score, ammo, slow-time bar, combo counter, and rank screen.	score, ammo_display, slow_bar, combo_count	draw_hud(), update_score(), show_combo(), show_rank_screen()
Camera	Tracks the player and applies smooth follow and screenshake to the viewport.	offset_x, offset_y, target, shake_intensity, shake_timer	follow(), apply_shake(), clamp_to_bounds(), update()

UML Diagram (Simplified Class Hierarchy):



3.3 Algorithms Involved

Algorithm / Technique	Where Used	Description
Finite State Machine (FSM)	Enemy AI (Python dict/class)	Each enemy cycles through Idle, Patrol, Alert, Chase, and Attack states. State transitions fire based on player distance, line-of-sight raycasts, and damage events. Implemented as Python methods with a state string attribute.
AABB Collision Detection	Player, Enemies, Bullets vs. Tiles	Axis-Aligned Bounding Box collision using pygame.Rect.colliderect(). Swept collision resolves overlaps per axis separately, preventing tunnelling at high bullet speeds.
Bullet Intercept Detection	BulletBlock class	Each frame, all active player bullets are tested against all active enemy bullets using distance-threshold collision. A successful intercept destroys both bullets and triggers a visual/score effect.
Pathfinding (BFS on tile grid)	GuardEnemy & SniperEnemy	Breadth-First Search on the tile grid finds a walkable path to the player when direct movement is blocked. Implemented with Python's collections.deque for the open queue.
Time Scaling	Slow-time mechanic	A global delta_time multiplier (0.25 during slow-time) is passed to all update() calls. Pygame's clock provides the base dt; the multiplier scales physics, animation, and bullet movement uniformly.
Score & Rank Formula	GameStats + UIManager	Score = (Base + kills*200 + blocks*300 - deaths*500) * time_bonus. Result is compared against fixed S/A/B/C/D threshold constants. Computed in Python at session end.
Event-Driven Input Queue	Player input handling	Pygame's event queue is polled each frame. Key-down events are pushed to an input buffer list, enabling jump and dash chaining across frames without missed inputs.

4. Statistical Data (Prop Stats)

4.1 Data Features

The following features are tracked per gameplay session using the GameStats class. Each feature accumulates at least 100 rows of data across test sessions:

shots_fired — Total number of bullets the player fires in a session. Type: Integer. Range: 0 – 999,999
bullet_blocks — Number of successful bullet-vs-bullet intercepts in a session. Type: Integer. Range: 0 – 999,999
accuracy_pct — Ratio of enemy-hitting shots to total shots fired. Type: Float 0.0–1.0. Range: 0 – 1
dash_count — Number of times the player activates the dash ability. Type: Integer. Range: 0 – 999,999
deaths — Number of times the player dies within a session. Type: Integer. Range: 0 – 999,999
kill_count — Number of enemies defeated per session. Type: Integer. Range: 0 – 100 (limit of enemies in all stage is not more than 100)
completion_time_ms — Milliseconds from level start to exit reached. Type: Integer. Range: 0 - 999,999
pixels_moved — Total pixel distance the player travels per session. Type: Integer. Range: 0 – 999,999
slow_time_uses — Number of times the slow-time ability was activated. Type: Integer. Range: 0 – 999,999
score — Final composite score computed at session end. Type: Integer. Range: 0 – 999,999

4.2 Data Recording Method

All data recording is handled inside the GameStats Python class, using only the Python standard library and Pygame — no external database is required:

• **Runtime accumulation:** GameStats methods (record_shot(), record_block(), record_dash(), record_death()) are called by game event handlers each time a relevant action occurs. All values are held in memory during the session.

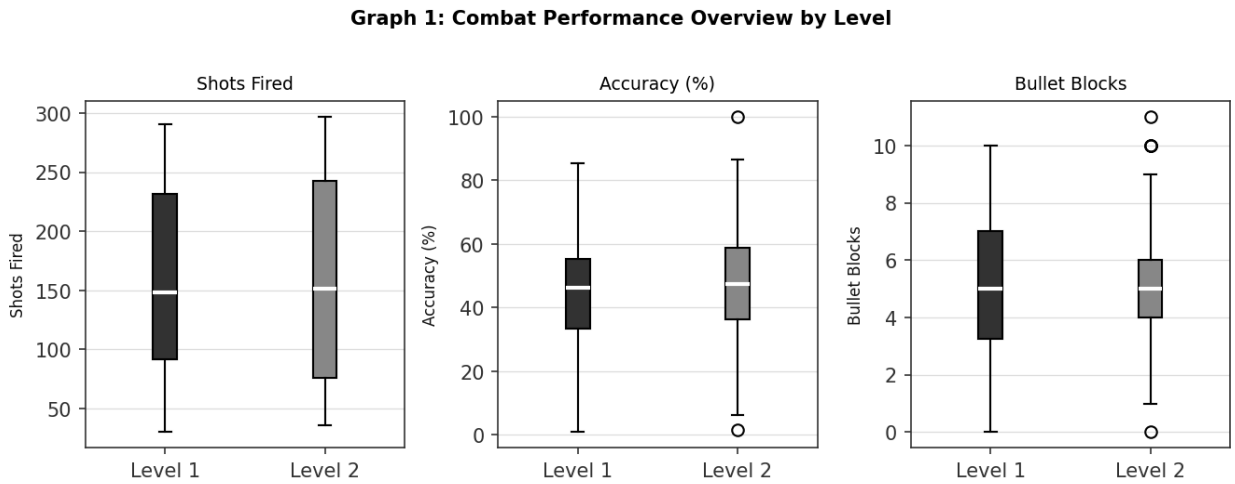
• **CSV export:** At session end, GameStats.export_csv() appends one row to sessions.csv using
Python's built-in csv module. Each row contains session_id, player_id, level_id, timestamp, and all 10 feature columns.

• **File structure:** A single sessions.csv file stores all sessions. The header row is written only if the file does not yet exist, so data accumulates safely across runs.
• **Reproducibility:** Each session is assigned a UUID generated by Python's uuid module, ensuring rows are uniquely identifiable for analysis. Target: 100 or more sessions per level collected through playtesting and repeated runs to ensure statistically meaningful sample sizes.

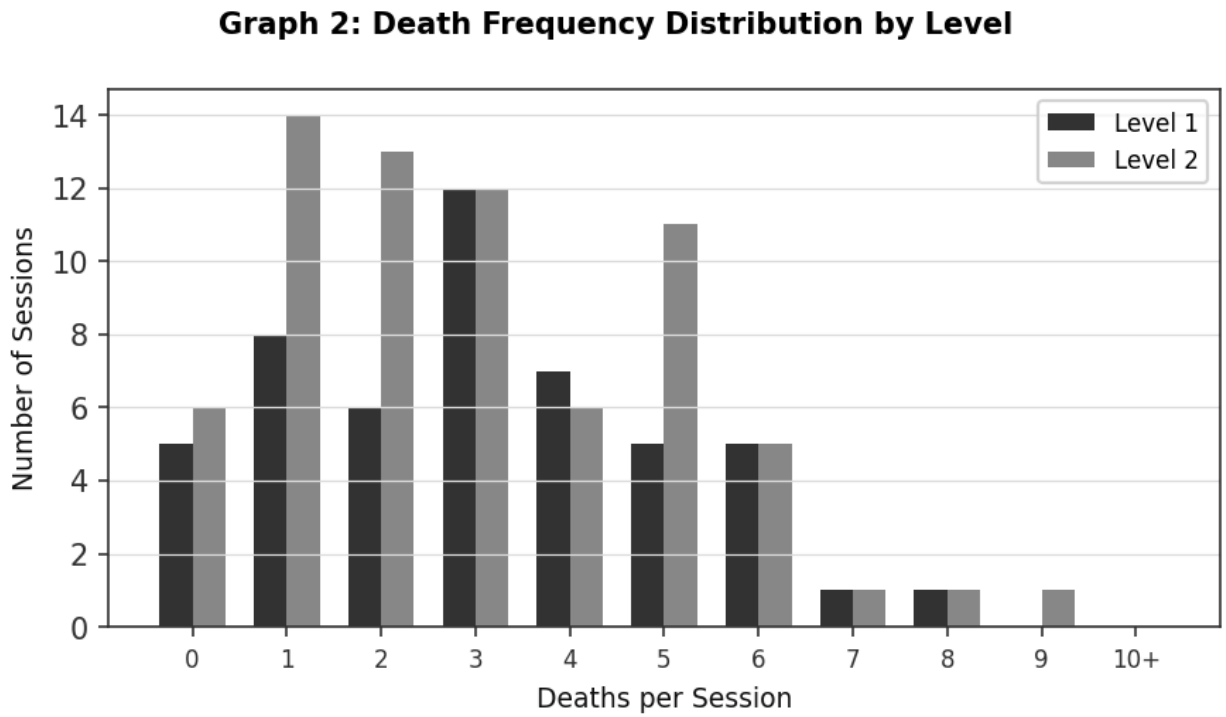
4.3 Data Analysis Report

All analysis is performed in Python using pandas, matplotlib, and seaborn, reading directly from sessions.csv:

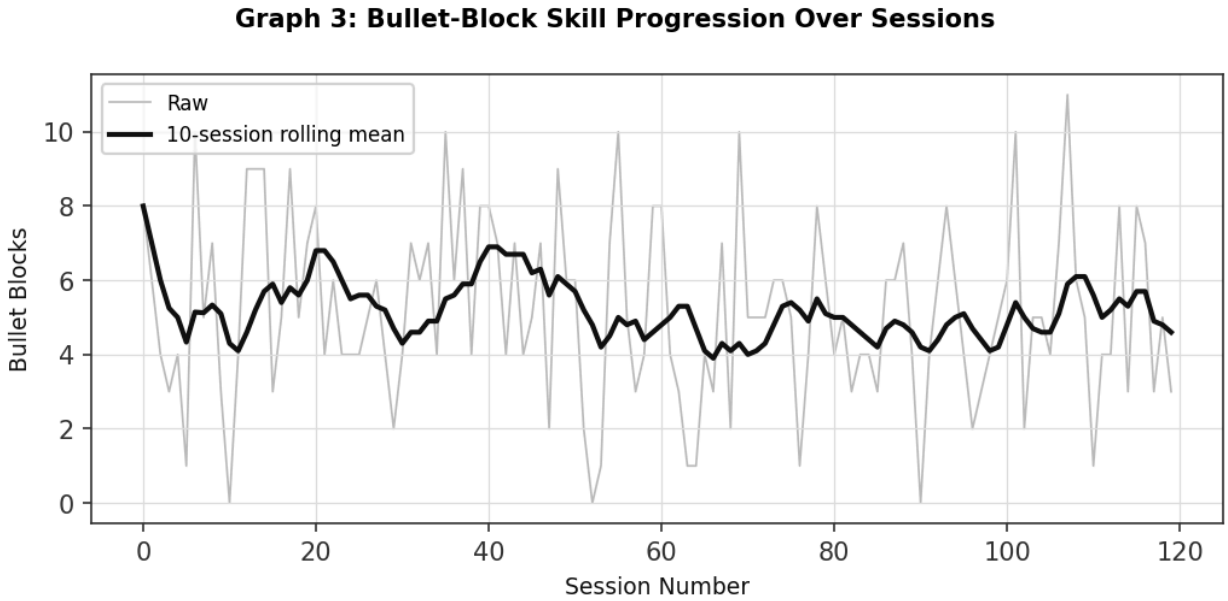
Analysis	Statistical Method	Tools & Implementation	Presentation
Combat performance overview	Mean, median, std of shots_fired, accuracy_pct, bullet_blocks	pandas: df.describe(), df.groupby().agg()seaborn: boxplot()	Box plots per level; summary statistics table
Death hotspot frequency	Frequency count of deaths grouped by checkpoint zone	pandas: df.groupby(['level_id','checkpoint']).sum()matplotlib: bar chart	Bar chart per level; heatmap overlay on tile map
Bullet-block skill progression	Rolling mean of bullet_blocks across sessions by timestamp	pandas: df.sort_values().rolling(10).mean()matplotlib: plot()	Line chart showing improvement over time
Accuracy vs. score correlation	Pearson correlation between accuracy_pct and score	pandas: df.corr()seaborn: regplot()	Scatter plot with regression line and r-value
Rank distribution	Frequency count and percentage of S/A/B/C/D ranks	pandas: df['rank'].value_counts(normalize=True)matplotlib: pie(), bar()	Pie chart and grouped bar chart by level
Dash & slow-time vs. survival	Descriptive stats of dash_count and slow_time_uses; correlation with deaths	pandas: df.describe(), df.corr()seaborn: histplot(), heatmap()	Histogram pair; correlation heatmap
Session completion rate	Ratio of sessions where player reached exit vs. total	pandas: df.groupby('level_id')['completed'].mean()	Bar chart of completion rate per level



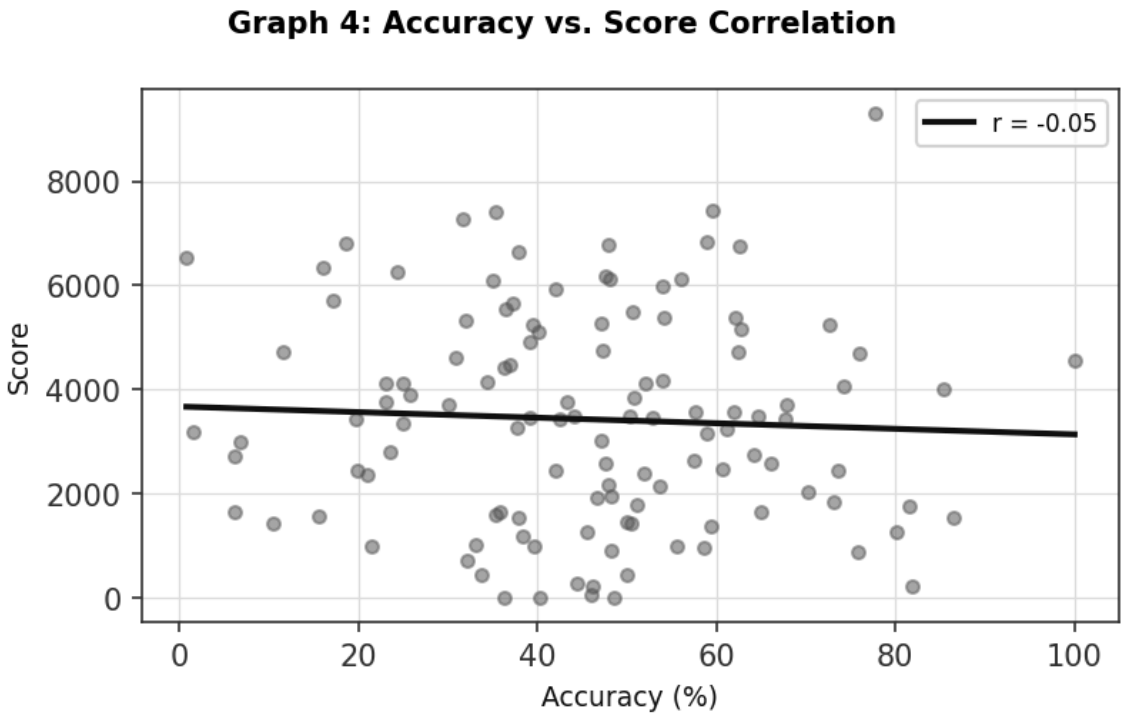
Graph 1: Box plots of shots fired, accuracy (%), and bullet blocks — split by level.



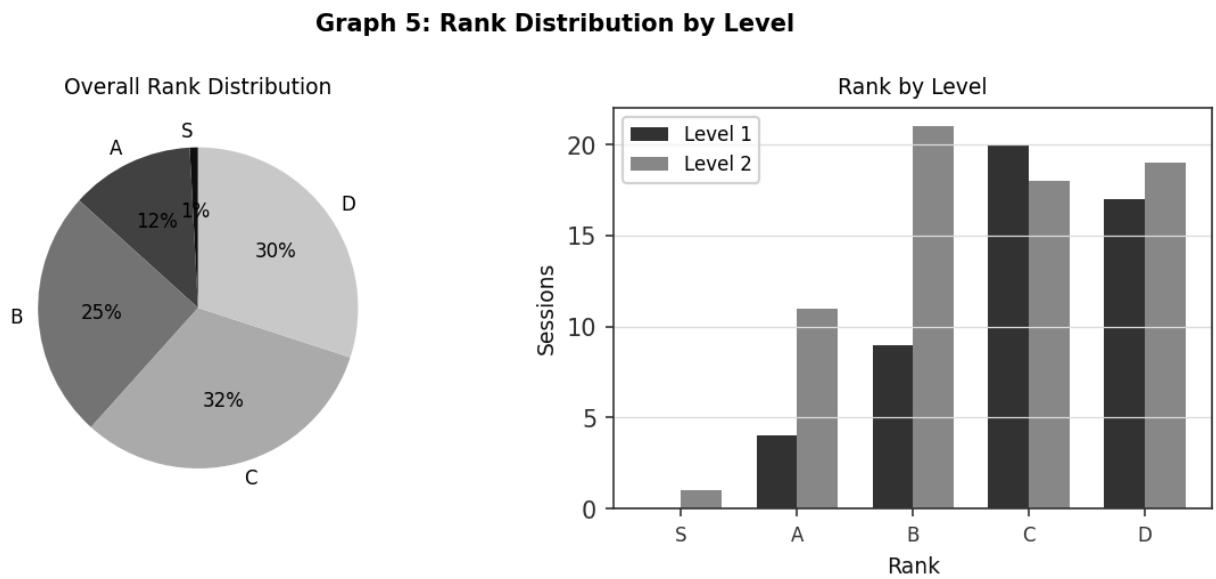
Graph 2: Death frequency distribution per session across both levels.



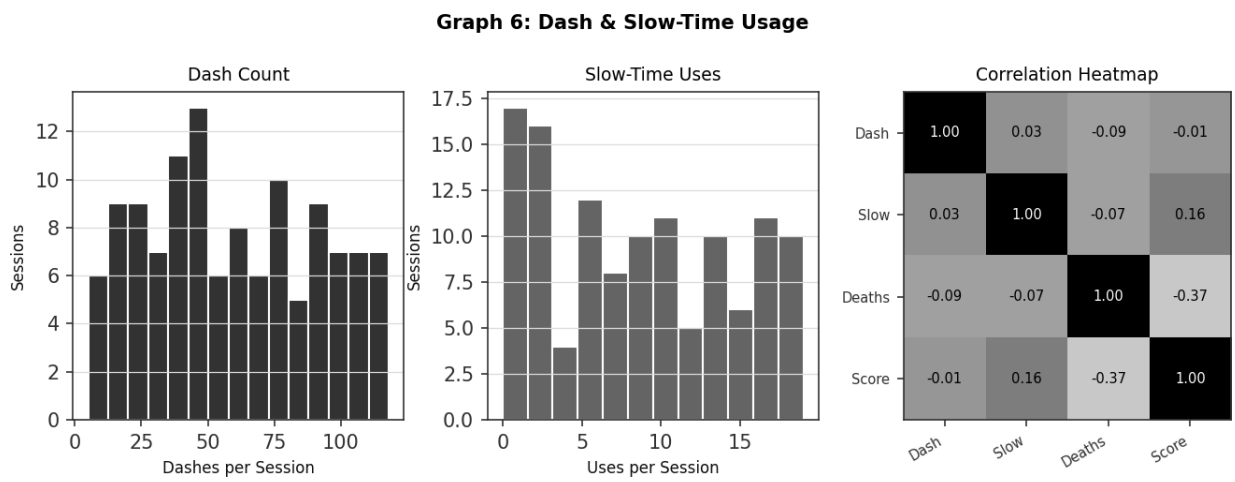
Graph 3: Bullet-block skill progression with 10-session rolling mean.



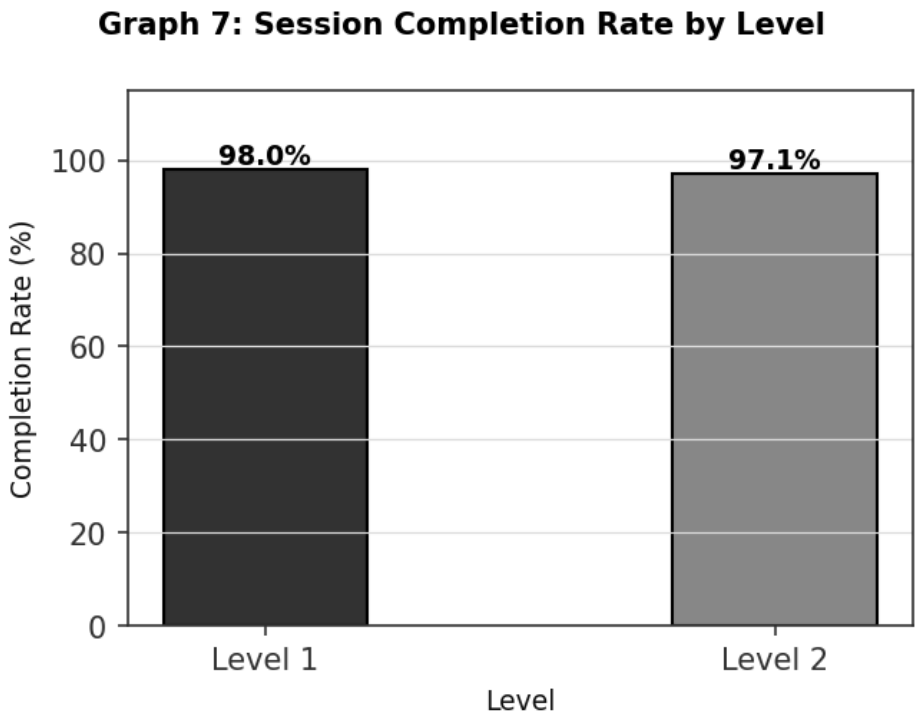
Graph 4: Scatter plot of accuracy (%) vs. score with linear regression line and r-value.



Graph 5: Rank distribution — overall pie chart and per-level bar chart.



Graph 6: Histograms of dash count and slow-time uses, plus correlation heatmap.



Graph 7: Session completion rate (%) per level.

5. Project Planning Timeline

Week	Week	Task	Task
1	8	Pre-Production & Design	GameStats
2	9	Core Engine (Python/Pygame)	CSV Export
3	10	Player Mechanics	Level 2 & Polish
4	11	Bullet Systems & Bullet-Block	Collect 100+ sessions
5	12	Enemy AI & FSM	analysis. Generate PNG charts and compile PDF

			report.
6	13	Level 1 Construction	Write final documentation
7	14	UI & Scoring	Final Submission

6. Document version

Version: 1.0
Date: 13 March 2025
Editor: Sirapat Pringprom (ID 6810545948)

Date	Name	Description of Revision, Feedback, Comments
14/3	Nattanan	Mostly formatting and a notice about project difficulty. You just need to fix the data part, if you are going to add range. Define it using what it COULD be, not what it should be. Even through that might mean it is 0 % or 100% Overall, good job, very detailed work 👍